In [6]:
```
import os
import sys

import matplotlib.colors as colors
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scanpy as sc
import seaborn as sns
import celloracle as co
co.__version__
import os, sys, shutil, importlib, glob
from tqdm.notebook import tqdm
%matplotlib inline
```

In [7]:
```
goi = "Maf_Pbx3"
save_folder = f"/xxx/analysis/PT_subset/perterb/figures_scMEGA_PERT_{goi}"
os.makedirs(save_folder, exist_ok=True)
save_folder
```

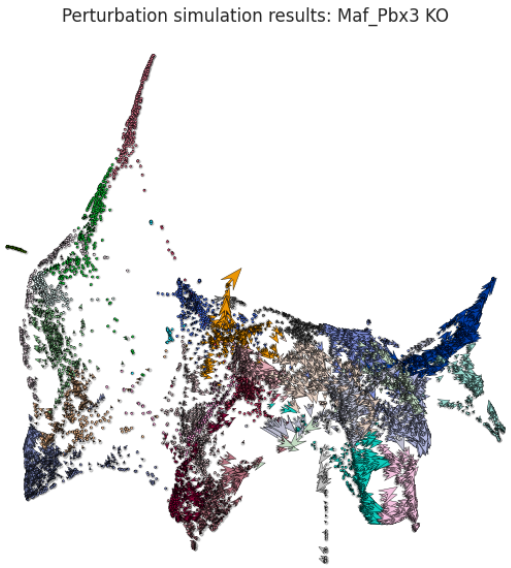
Out[7]:
```
'/xxx/analysis/PT_subset/perterb/figures_scMEGA_PERT_Maf_Pbx3'
```

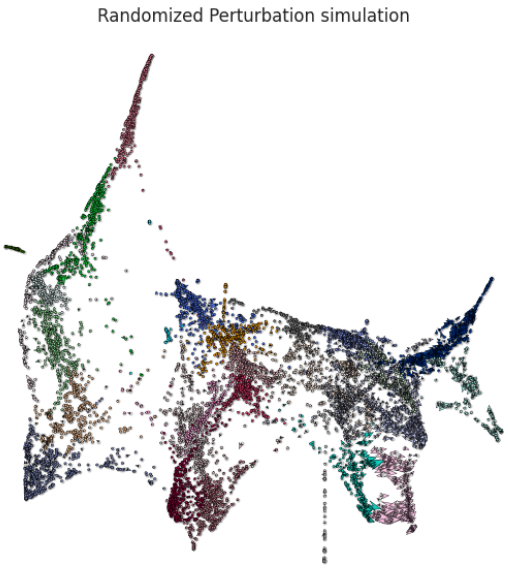
In [8]:
```
oracle = co.load_hdf5("/xxx/analysis/PT_subset/Sobj_PT1_2_inj_step4.celloracle.orac
```

In [9]:
```
# Enter perturbation conditions to simulate signal propagation after the perturbati
oracle.simulate_shift(perturb_condition={"Maf": 0.0,"Pbx3": 0.0},
                      n_propagation=3)
#https://compscbio.github.io/docs/html/day/21_Simulation_analysis.html
```

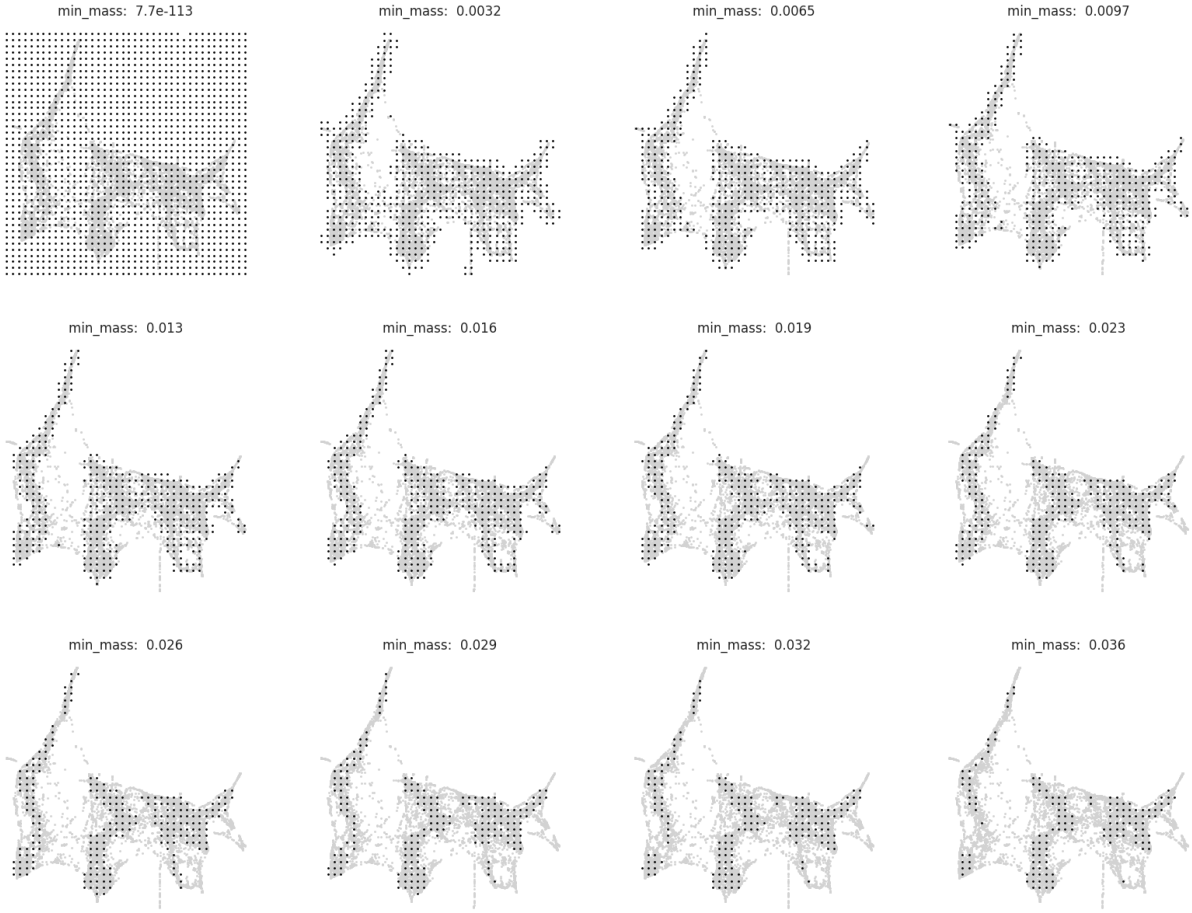
In [10]:
```
oracle.estimate_transition_prob(n_neighbors=200, knn_random=True, sampled_fraction=

# Calculate embedding
oracle.calculate_embedding_shift(sigma_corr = 0.05)
```

In [11]:
```
oracle.to_hdf5(f"{save_folder}/{goi}_step1.celloracle.oracle")
fig, ax = plt.subplots(1, 2,  figsize=[15, 7])

scale = 40 # Show quiver plot
oracle.plot_quiver(scale=scale, ax=ax[0])
ax[0].set_title(f"Perturbation simulation results: {goi} KO")

# Show quiver plot that was calculated with randomized transition probability.
oracle.plot_quiver_random(scale=scale, ax=ax[1])
ax[1].set_title(f"Randomized Perturbation simulation")


plt.savefig(f"{save_folder}/{goi}_Quiver.pdf")
plt.savefig(f"{save_folder}/{goi}_Quiver.png")
plt.show()
```





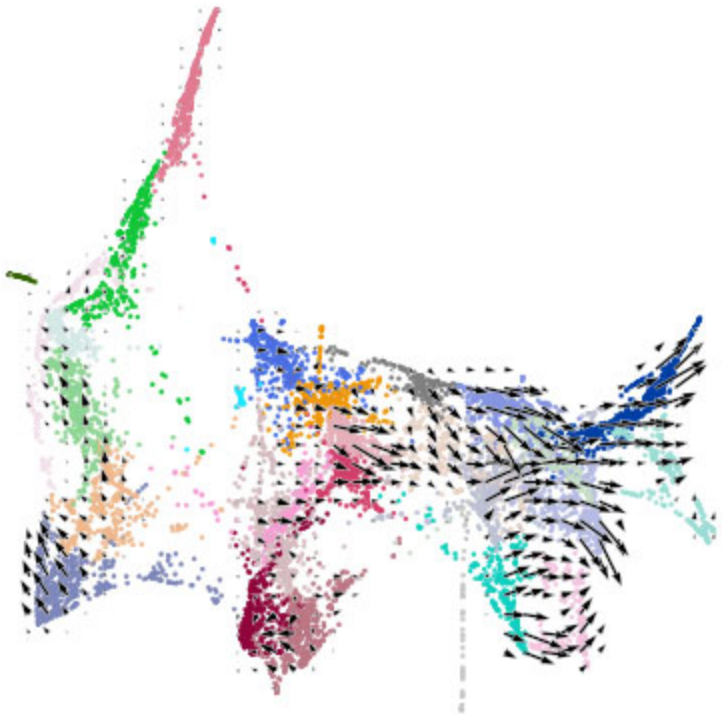
In [12]:
```
# 4.2. Vector field graph

# We will visualize the simulation results as a vector field on a digitized grid. S
# 4.2.1 Find parameters for n_grid and min_mass

# n_grid: Number of grid points.

# min_mass: Threshold value for the cell density. The appropriate values for these

# n_grid = 40 is a good starting value.
n_grid = 40
oracle.calculate_p_mass(smooth=0.8, n_grid=n_grid, n_neighbors=200)

#Please run oracle.suggest_mass_thresholds() to display a range of min_mass paramet

# Search for best min_mass.
oracle.suggest_mass_thresholds(n_suggestion=12)

plt.savefig(f"{save_folder}/{goi}_{n_grid}_n_grid.pdf")
plt.savefig(f"{save_folder}/{goi}_{n_grid}_n_grid.png")
plt.show()
```



In [13]:
```
min_mass = 0.01
oracle.calculate_mass_filter(min_mass=min_mass, plot=True)
plt.savefig(f"{save_folder}/{goi}_{min_mass}_n_grid.pdf")
plt.show()
```

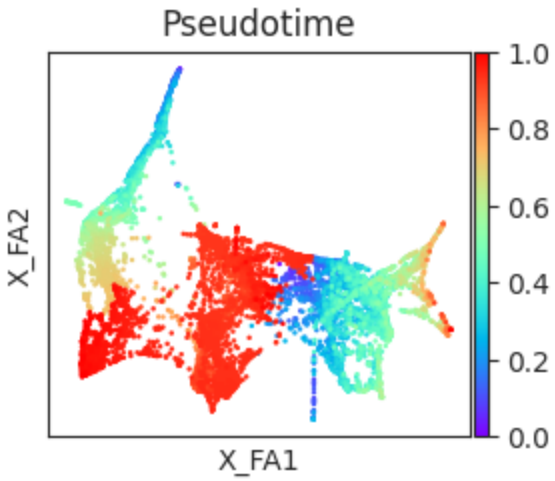
In [14]:
```
# 4.2.2 Plot vector fields

#     Again, we need to adjust the scale parameter. Please seek to find the optimal

#     If you don't see any vector, you can try the smaller scale parameter to magni

fig, ax = plt.subplots(1, 2,  figsize=[13, 6])

scale_simulation = 0.5
# Show quiver plot
oracle.plot_simulation_flow_on_grid(scale=scale_simulation, ax=ax[0])
ax[0].set_title(f"Simulated cell identity shift vector: {goi} KO")

# Show quiver plot that was calculated with randomized graph.
oracle.plot_simulation_flow_random_on_grid(scale=scale_simulation, ax=ax[1])
ax[1].set_title(f"Randomized simulation vector")

plt.savefig(f"{save_folder}/{goi}_SimCellIdentityShift.pdf")
plt.savefig(f"{save_folder}/{goi}_SimCellIdentityShift.png")
plt.show()
```

In [15]:
```
fig, ax = plt.subplots(figsize=[5, 5])

oracle.plot_cluster_whole(ax=ax, s=2)
oracle.plot_simulation_flow_on_grid(scale=scale_simulation, ax=ax, show_background=
plt.savefig(f"{save_folder}/{goi}_PlotVectorcolors.pdf")
plt.savefig(f"{save_folder}/{goi}_PlotVectorcolors.png")
plt.show()
```

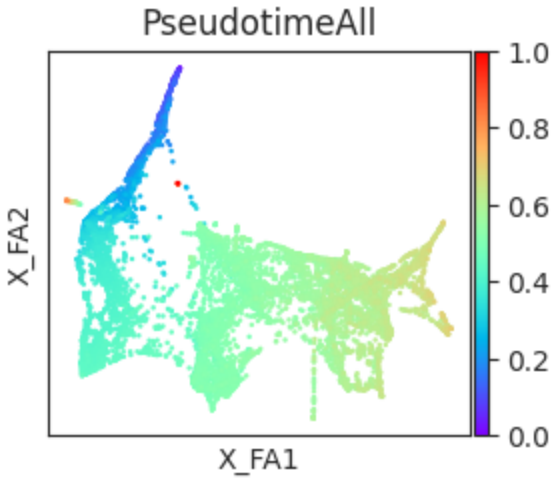


In [16]:
```
# Visualize pseudotime
fig, ax = plt.subplots(figsize=[3,2.5])

sc.pl.embedding(adata=oracle.adata, basis=oracle.embedding_name, ax=ax,cmap="rainbo
                color=["Pseudotime"])
```

```
plt.savefig(f"{save_folder}/{goi}_Trajectory.pdf")

plt.show()

# Visualize pseudotime
fig, ax = plt.subplots(figsize=[3,2.5])

sc.pl.embedding(adata=oracle.adata, basis=oracle.embedding_name, ax=ax,cmap="rainbo
                color=["PseudotimeAll"])
plt.savefig(f"{save_folder}/{goi}_Trajectory_All.pdf")

plt.show()
```



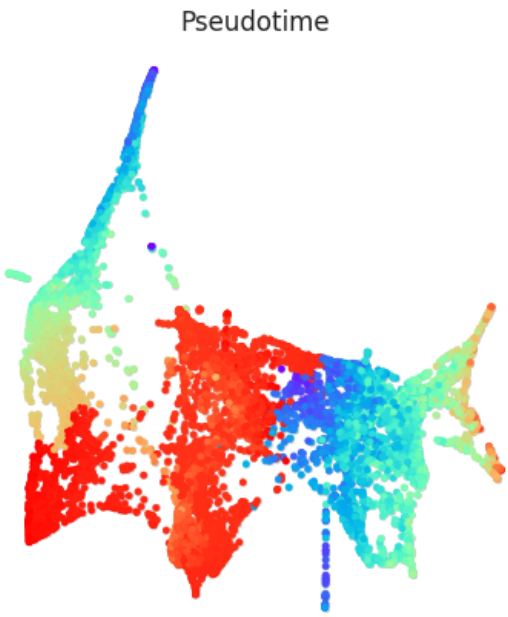

```
<Figure size 640x480 with 0 Axes>
```



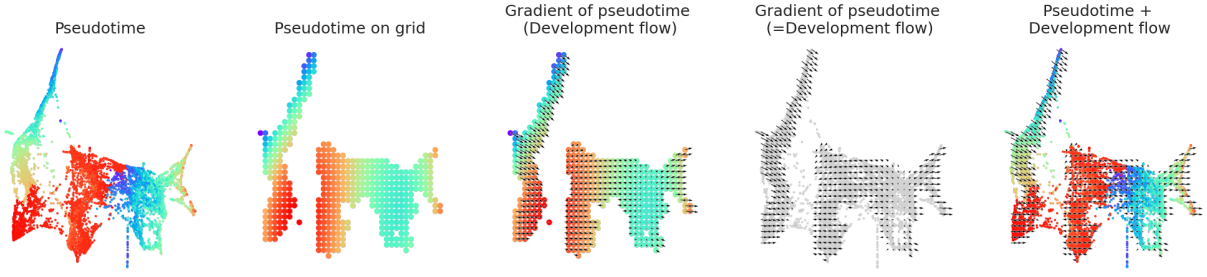

```
<Figure size 640x480 with 0 Axes>
```

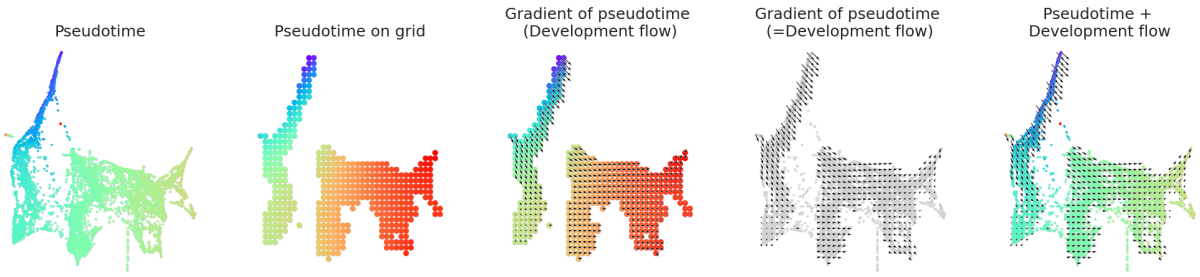
In [17]:
```
#5.3. Make Gradient_calculator object
from celloracle.applications import Gradient_calculator

# Instantiate Gradient calculator object
gradient = Gradient_calculator(oracle_object=oracle, pseudotime_key="Pseudotime")
# We need to set n_grid and min_mass for the pseudotime grid point calculation too.
# We already know approproate values for them. Please set the same values as step 4

gradient.calculate_p_mass(smooth=.8, n_grid=n_grid, n_neighbors=200)
gradient.calculate_mass_filter(min_mass=min_mass, plot=False)
```

```
#for only one group "ALL"
gradient2 = Gradient_calculator(oracle_object=oracle, pseudotime_key="PseudotimeAll
# We need to set n_grid and min_mass for the pseudotime grid point calculation too.
# We already know approproate values for them. Please set the same values as step 4

gradient2.calculate_p_mass(smooth=.8, n_grid=n_grid, n_neighbors=200)
gradient2.calculate_mass_filter(min_mass=min_mass, plot=False)
# 5.4 Transfer pseudotime values to the grid points.

# Next, we convert the pseudotime data into grid points. For this calculation we ca

#     knn: K-Nearesr Neighbor regression. You will need to set number of neighbors.
#Please adjust n_knn for best results.This will depend on the population size and d

# gradient.transfer_data_into_grid(args={"method": "knn", "n_knn":50})

#     polynomial: Polynomial regression using x-axis and y-axis of dimensional redu

# In general, this method will be more robust. Please use this method if knn method
#n_poly is the number of degrees for the polynomial regression model.
#Please try to find appropriaten_poly searching for best results.

gradient.transfer_data_into_grid(args={"method": "polynomial", "n_poly":3}, plot=Tr
plt.savefig(f"{save_folder}/{goi}_Pseudotime_PGrid.pdf")
plt.savefig(f"{save_folder}/{goi}_Pseudotime_PGrid.png")
plt.show()

gradient2.transfer_data_into_grid(args={"method": "polynomial", "n_poly":3}, plot=T
plt.savefig(f"{save_folder}/{goi}_PseudotimeAll_PGrid.pdf")
plt.savefig(f"{save_folder}/{goi}_PseudotimeAll_PGrid.png")
plt.show()
```

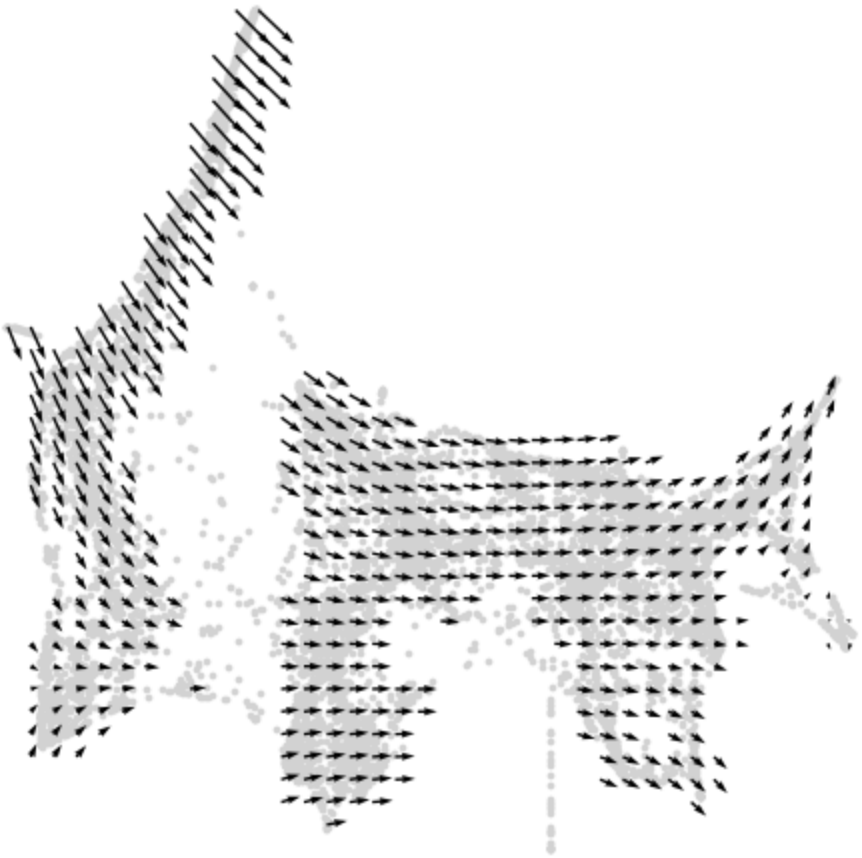


In [18]:
```
plt.rcParams.update({'font.size': 15})

# 5.5. Calculate Gradient vectors

# Calculate the 2D vector map to represents the pseudotime gradient. After the grad
#of the vector will be normalized automatically.

# Please adjust the scale parameter to adjust vector length visualization.

# Calculate graddient
gradient.calculate_gradient()
gradient2.calculate_gradient()

# Show results
scale_dev = 35
gradient.visualize_results(scale=scale_dev, s=5)


plt.savefig(f"{save_folder}/{goi}_Pseudotime_DevelopmentFlow.pdf")
plt.savefig(f"{save_folder}/{goi}_Pseudotime_DevelopmentFlow.png")
plt.show()

gradient2.visualize_results(scale=scale_dev, s=5)
plt.savefig(f"{save_folder}/{goi}_PseudotimeAll_DevelopmentFlow.pdf")
plt.savefig(f"{save_folder}/{goi}_PseudotimeAll_DevelopmentFlow.png")
plt.show()
```





In [19]:
```
# Visualize results
fig, ax = plt.subplots(figsize=[6, 6])
gradient.plot_dev_flow_on_grid(scale=scale_dev, ax=ax)

plt.savefig(f"{save_folder}/{goi}_DevelopmentFlow.pdf")
plt.savefig(f"{save_folder}/{goi}_DevelopmentFlow.png")
plt.show()

# Visualize results
fig, ax = plt.subplots(figsize=[6, 6])
gradient2.plot_dev_flow_on_grid(scale=scale_dev, ax=ax)

plt.savefig(f"{save_folder}/{goi}_DevelopmentFlow_All.pdf")
plt.savefig(f"{save_folder}/{goi}_DevelopmentFlow_All.png")
plt.show()
```



In [20]:
```
# 5.6. Calculate Inner product between two vectors

# We will use the inner product calculations to quantitatively compare the 2D devel

#     If you are not familiar with Inner product / Dot product, please see https://

#     The inner product represents the similarity between two vectors.

#     Using the inner product, we compare the 2D vector field of perturbation simul

#     The inner product will be a positive value when two vectors are pointing in t

#     Conversely, the inner product will be a negative value when two vectors are p

#     The length of the vectors also affects the magnitude of inner product value.

# In summary, we quantitatively compare the directionality and size of vectors betw

#     A negative PS means that the TF perturbation would block differentiation .

#     A positive PS means that the TF perturbation would promote differentiation .
from celloracle.applications import Oracle_development_module

# Make Oracle_development_module to compare two vector field
dev = Oracle_development_module()
```

```
# Load development flow
dev.load_differentiation_reference_data(gradient_object=gradient)

# Load simulation result
dev.load_perturb_simulation_data(oracle_object=oracle)


# Calculate inner produc scores
dev.calculate_inner_product()
dev.calculate_digitized_ip(n_bins=10)

#All
# Make Oracle_development_module to compare two vector field
dev2 = Oracle_development_module()

# Load development flow
dev2.load_differentiation_reference_data(gradient_object=gradient2)

# Load simulation result
dev2.load_perturb_simulation_data(oracle_object=oracle)


# Calculate inner produc scores
dev2.calculate_inner_product()
dev2.calculate_digitized_ip(n_bins=10)
```

In [21]:

```
plt.rcParams.update({'font.size': 10})

# 5.7. Show results

#     Here, need to adjust the vm parameter for the PS score color visualization.
#Please seek to find the optimal vm parameter that provides good visualization.

# If you don't see any color in the left panel below, you can try
#the smaller vm parameter to magnify the scale of vm visualization.
#However, if you see colors in the randomized results (right panel), it means the v

# Show perturbation scores
vm = 0.015 #0.02

fig, ax = plt.subplots(1, 2, figsize=[12, 6])
dev.plot_inner_product_on_grid(vm=0.01, s=50, ax=ax[0])
ax[0].set_title(f"PS")

dev.plot_inner_product_random_on_grid(vm=vm, s=50, ax=ax[1])
ax[1].set_title(f"PS calculated with Randomized simulation vector")


plt.savefig(f"{save_folder}/{goi}_PS_calculated_with_Randomized.pdf")
plt.savefig(f"{save_folder}/{goi}_PS_calculated_with_Randomized.png") #perturbation
plt.show()

#All
vm = 0.015 #0.02

fig, ax = plt.subplots(1, 2, figsize=[12, 6])
```

```
dev2.plot_inner_product_on_grid(vm=0.01, s=50, ax=ax[0])
ax[0].set_title(f"PS_All")

dev2.plot_inner_product_random_on_grid(vm=vm, s=50, ax=ax[1])
ax[1].set_title(f"PS_All calculated with Randomized simulation vector")


plt.savefig(f"{save_folder}/{goi}_PS_All_calculated_with_Randomized.pdf")
plt.savefig(f"{save_folder}/{goi}_PS_All_calculated_with_Randomized.png") #perturba
plt.show()
```

PS

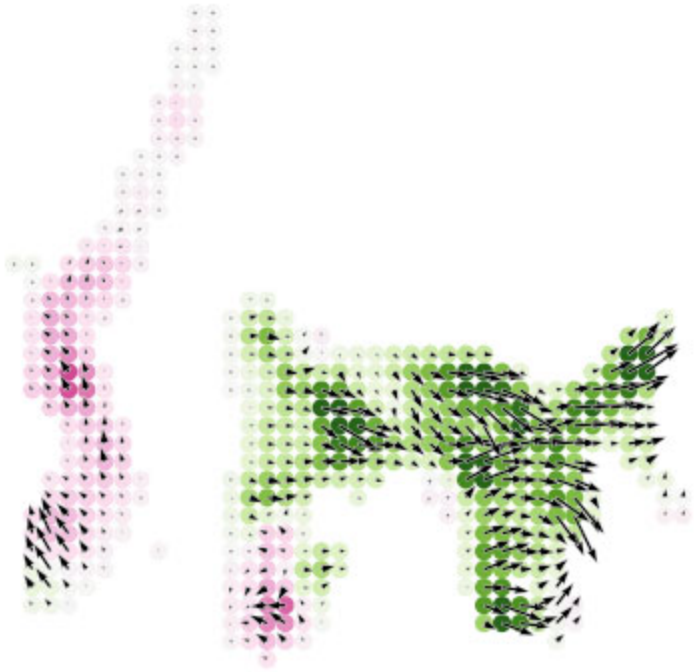
PS calculated with Randomized simulation vector

PS_All

PS_All calculated with Randomized simulation vector

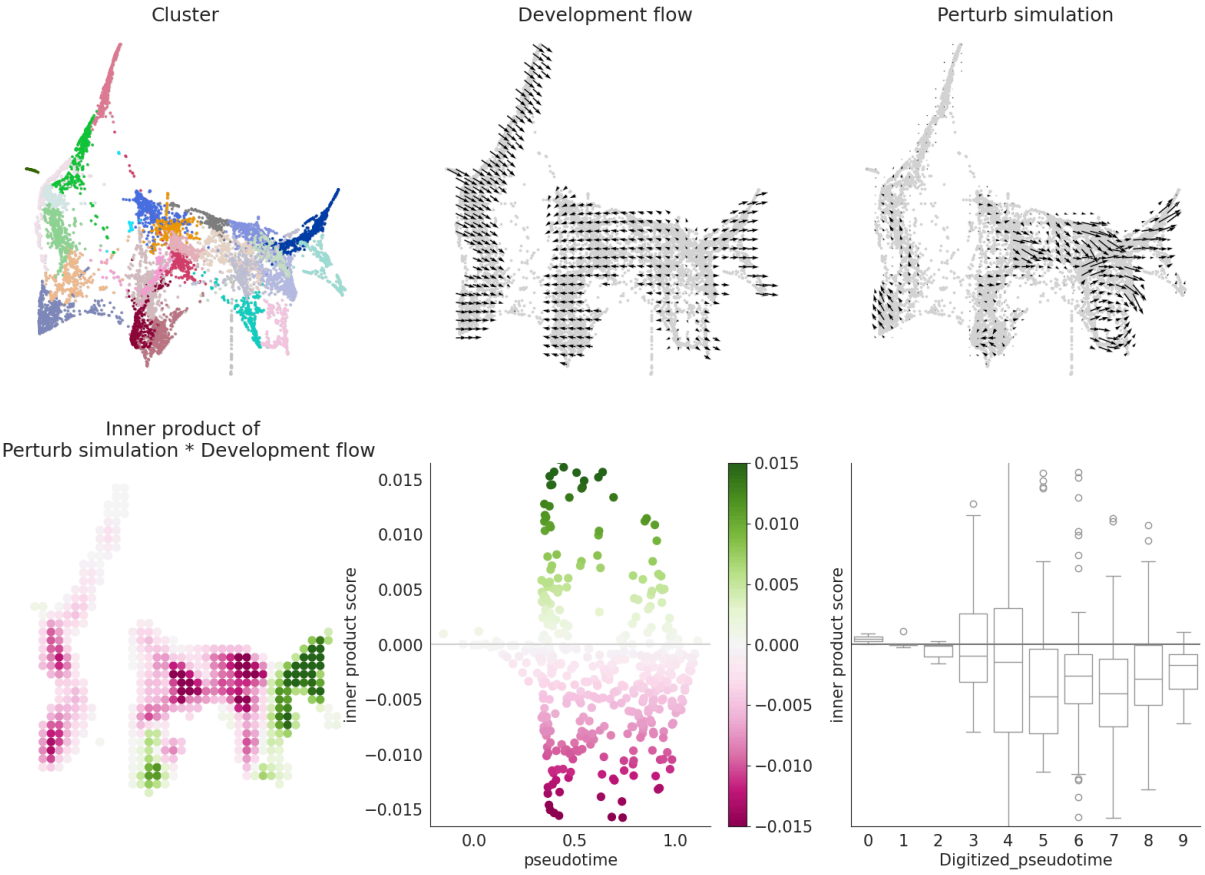
In [22]:
```
# Show perturbation scores with perturbation simulation vector field
fig, ax = plt.subplots(figsize=[5, 5])
dev.plot_inner_product_on_grid(vm=vm, s=30, ax=ax)


dev.plot_simulation_flow_on_grid(scale=scale_simulation, show_background=False, ax=
```

```
plt.savefig(f"{save_folder}/{goi}_PS_in_FlowDevelopmentSimulated.pdf")
plt.savefig(f"{save_folder}/{goi}_PS_in_FlowDevelopmentSimulated.png")

#All
# Show perturbation scores with perturbation simulation vector field
fig, ax = plt.subplots(figsize=[5, 5])
dev2.plot_inner_product_on_grid(vm=vm, s=30, ax=ax)


dev2.plot_simulation_flow_on_grid(scale=scale_simulation, show_background=False, ax

plt.savefig(f"{save_folder}/{goi}_PS_All_in_FlowDevelopmentSimulated.pdf")
plt.savefig(f"{save_folder}/{goi}_PS_All_in_FlowDevelopmentSimulated.png")
```



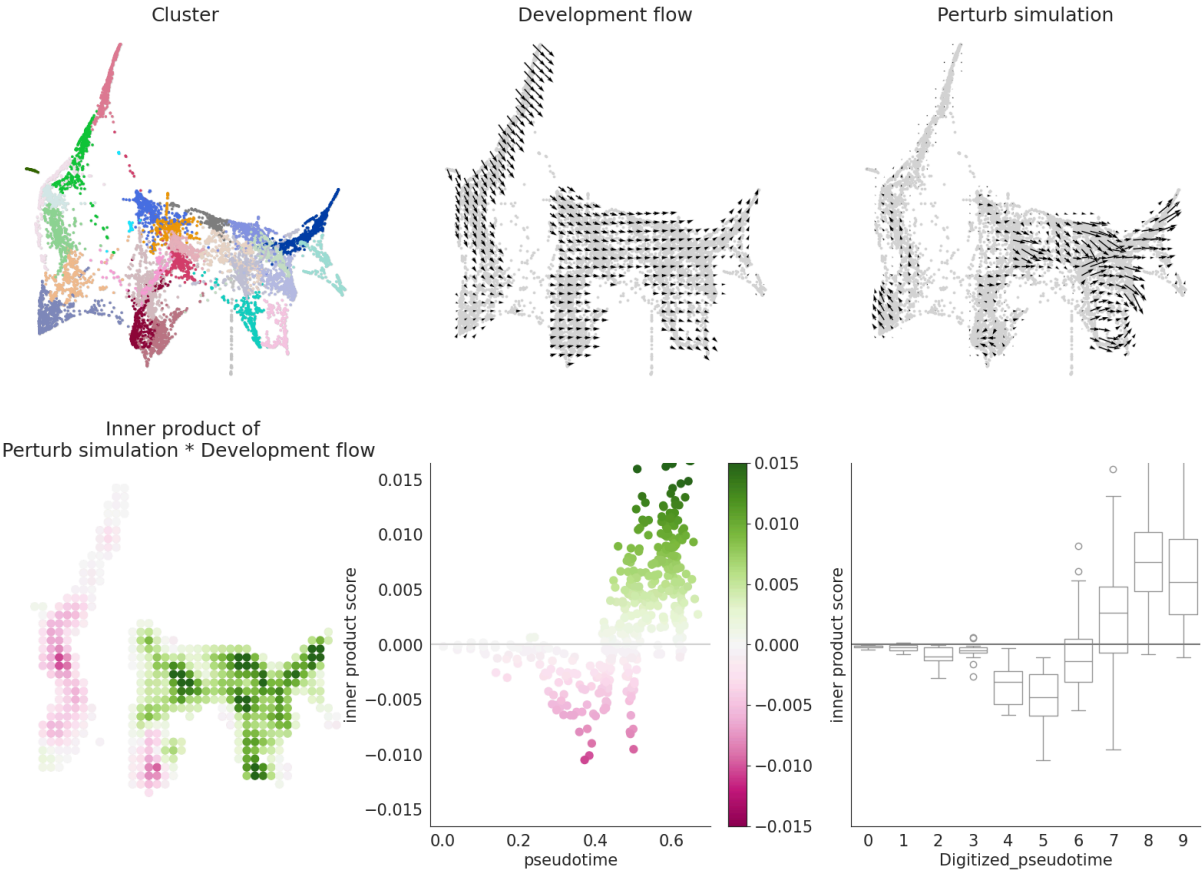
In [23]:
```
# Let's visualize the results
#For separate pseudotime
plt.rcParams.update({'font.size': 15})

dev.visualize_development_module_layout_0(s=5,
                                          scale_for_simulation=scale_simulation,
                                          s_grid=50,
                                          scale_for_pseudotime=scale_dev,
                                          vm=vm)
plt.savefig(f"{save_folder}/{goi}_FinalResult.pdf")
plt.savefig(f"{save_folder}/{goi}_FinalResult.png")
```

```
2024-08-15 14:05:58,653 - INFO - Using categorical units to plot a list of strings t
hat are all parsable as floats or dates. If these strings should be plotted as numbe
rs, cast to the appropriate data type before plotting.
2024-08-15 14:05:58,671 - INFO - Using categorical units to plot a list of strings t
hat are all parsable as floats or dates. If these strings should be plotted as numbe
rs, cast to the appropriate data type before plotting.
```

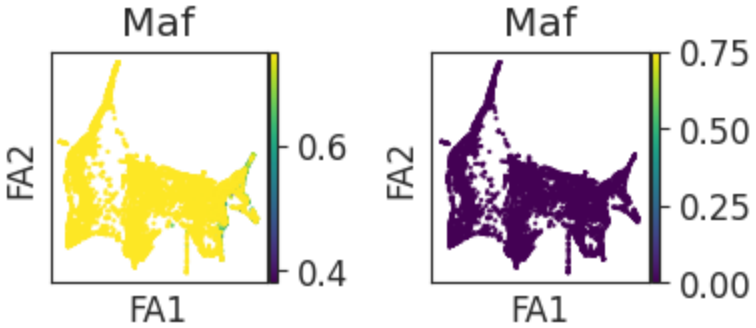


In [24]:
```
# Let's visualize the results
plt.rcParams.update({'font.size': 15})

dev2.visualize_development_module_layout_0(s=5,
                                           scale_for_simulation=scale_simulation,
                                           s_grid=50,
                                           scale_for_pseudotime=scale_dev,
                                           vm=vm)
plt.savefig(f"{save_folder}/{goi}_FinalResult_All.pdf")
plt.savefig(f"{save_folder}/{goi}_FinalResult_All.png")
```

```
2024-08-15 14:06:10,953 - INFO - Using categorical units to plot a list of strings t
hat are all parsable as floats or dates. If these strings should be plotted as numbe
rs, cast to the appropriate data type before plotting.
2024-08-15 14:06:10,963 - INFO - Using categorical units to plot a list of strings t
hat are all parsable as floats or dates. If these strings should be plotted as numbe
rs, cast to the appropriate data type before plotting.
```



In [25]:
```
plt.rcParams.update({'font.size': 12})

ge="Maf"
row=0
# The mv value change  for each gene is the max expression between after and before
mv=.75
# The values for Figure 6D: ELF3 mv=.75
#                 Figure 6H: TPM1 mv=2
#                            VCAM1 mv=.75
#                            ITGB8 mv=3
fig, ax = plt.subplots(1, 2,  figsize=[4,1.5], gridspec_kw={'wspace':0.6})
sc.pl.draw_graph(oracle.adata,color=ge,vmax=mv,
                  layer="imputed_count", use_raw=False, ax=ax[0], cmap="viridis", s
sc.pl.draw_graph(oracle.adata, color=ge,vmax=mv,
                 layer="simulated_count", use_raw=False,ax=ax[1], cmap="viridis", s
plt.show()
```



In [26]:
```
## Perform differential expression analysis PT PT injured before and after KO
sc.tl.rank_genes_groups(oracle.adata, groupby='celltype', layer="imputed_count",met
#The logfoldchanges are now stored in the adata object
result = oracle.adata.uns['rank_genes_groups']
groups = result['names'].dtype.names
groups
df1 = pd.DataFrame({group+'' + key:result[key][group] for group in groups for key i

sc.tl.rank_genes_groups(oracle.adata, groupby='celltype', layer="simulated_count",m
#The logfoldchanges are now stored in the adata object
result = oracle.adata.uns['rank_genes_groups']
groups = result['names'].dtype.names
groups

df2 = pd.DataFrame({group+'' + key:result[key][group] for group in groups for key i
```

In [27]:
```
# Define the conditions
groups = ["PT_S1", "PT_S2", "inj_PT_S1_S2"]

# Initialize an empty list to store the results for each group
all_dfs = []

# Loop over each group to perform the differential expression analysis and merge
for group in groups:
    # Extract data for the current group from df1 and df2
    d1 = df1[[f'{group}names', f'{group}logfoldchanges', f'{group}pvals_adj']]
    d2 = df2[[f'{group}names', f'{group}logfoldchanges', f'{group}pvals_adj']]

    # Merge the dataframes on the gene names
    d3 = pd.merge(d1, d2, on=f'{group}names')

    # Filter for genes where logfold changes differ between the two datasets
    d3 = d3[(d3[f"{group}logfoldchanges_x"] != d3[f"{group}logfoldchanges_y"])]

    # Filter for significantly expressed genes
    d3 = d3[(d3[f"{group}pvals_adj_x"] < 0.05)]

    # Add a column to indicate which group has the higher logfold change
    d3['UP_DEG'] = 'other'
    d3.loc[d3[f'{group}logfoldchanges_x'] > 0, 'UP_DEG'] = group

    # Calculate the absolute difference in logfold changes
    d3['abs_diff'] = abs(d3[f"{group}logfoldchanges_x"] - d3[f"{group}logfoldchange

    # Sort the dataframe by the absolute difference in logfold changes
    d3 = d3.sort_values(by=['abs_diff'], ascending=False)

    # Split the data into those with positive and negative differences
    d4_up = d3[d3['UP_DEG'] == group].sort_values(by=['abs_diff'], ascending=False)
    d4_down = d3[d3['UP_DEG'] != group].sort_values(by=['abs_diff'], ascending=True

    # Concatenate and append to the list of all results
    df = pd.concat([d4_down, d4_up])
    all_dfs.append(df)
```

```
# Concatenate all group results into a final dataframe
final_df = pd.concat(all_dfs, ignore_index=True)

# Optional: Save or display the dataframe
# final_df.to_csv(f"{save_folder}/dotplot_combined_groups.csv")
final_df
```

Out[27]:

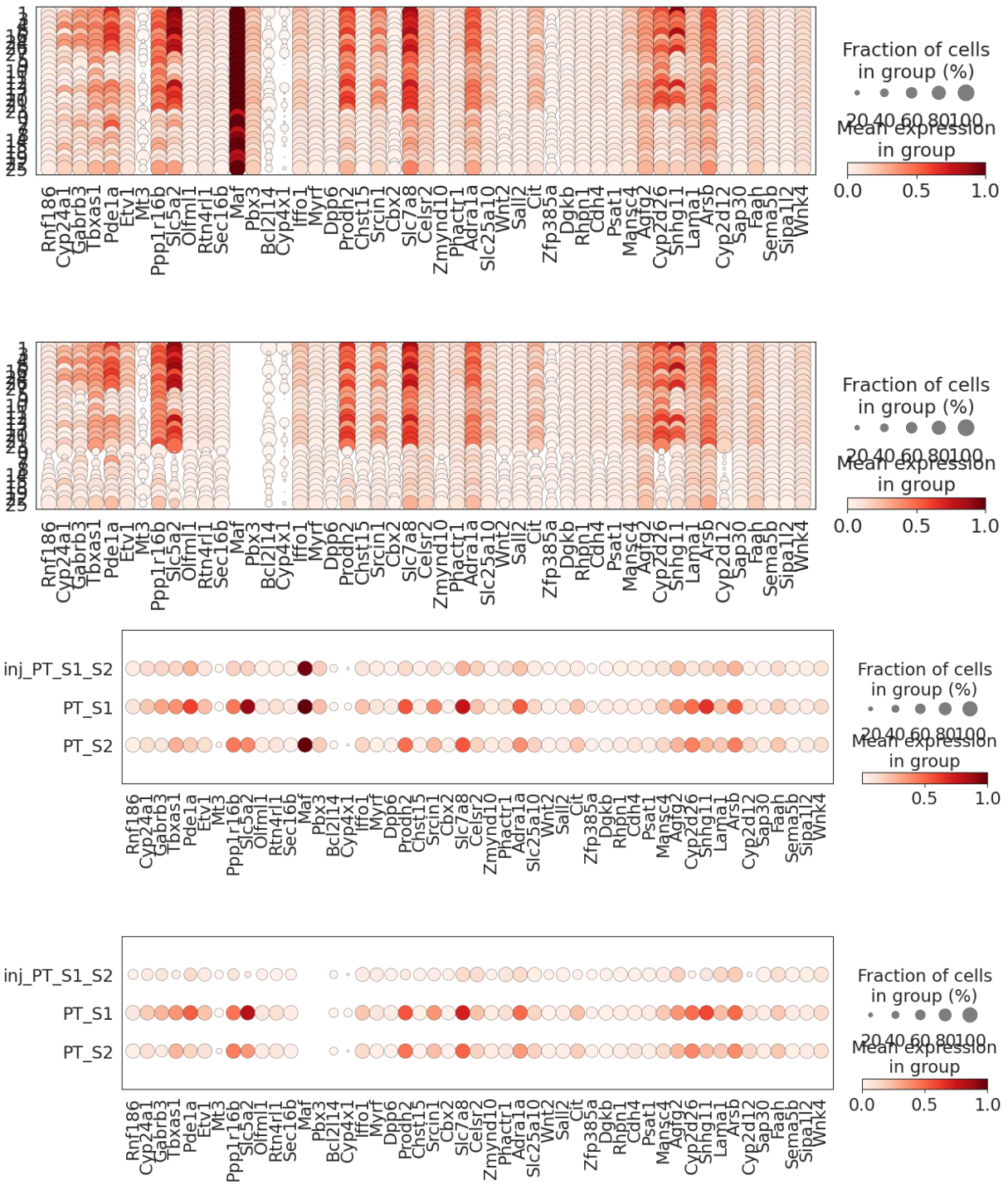
	PT_S1names	PT_S1logfoldchanges_x	PT_S1pvals_adj_x	PT_S1logfoldchanges_y	PT_S1
0	Sostdc1	-0.182735	1.920588e-47	-0.182735	1
1	Tmem44	-0.902729	8.421519e-58	-0.902728	7
2	Pcdhgb6	-0.429330	2.542443e-05	-0.429327	1
3	Bsn	-1.068014	6.514742e-116	-1.068009	2.5
4	Sprn	-1.325579	9.020630e-05	-1.325566	6
...	...	...	...	...	
3836	NaN	NaN	NaN	NaN	
3837	NaN	NaN	NaN	NaN	
3838	NaN	NaN	NaN	NaN	
3839	NaN	NaN	NaN	NaN	
3840	NaN	NaN	NaN	NaN	

3841 rows × 17 columns

In [28]:

```
# Define the number of top genes to select for each group
num_top_genes = 49

# Initialize dictionaries to store the marker genes for each group
marker_genes_dict = {}

# Loop over each group to populate the marker_genes_dict
for group in groups:
    # Select the relevant DataFrame for the current group
    df_group = final_df[final_df['UP_DEG'] == group]

    # Sort and select the top genes based on absolute difference
    top_genes = df_group.sort_values(by=['abs_diff'], ascending=False).head(num_top
```

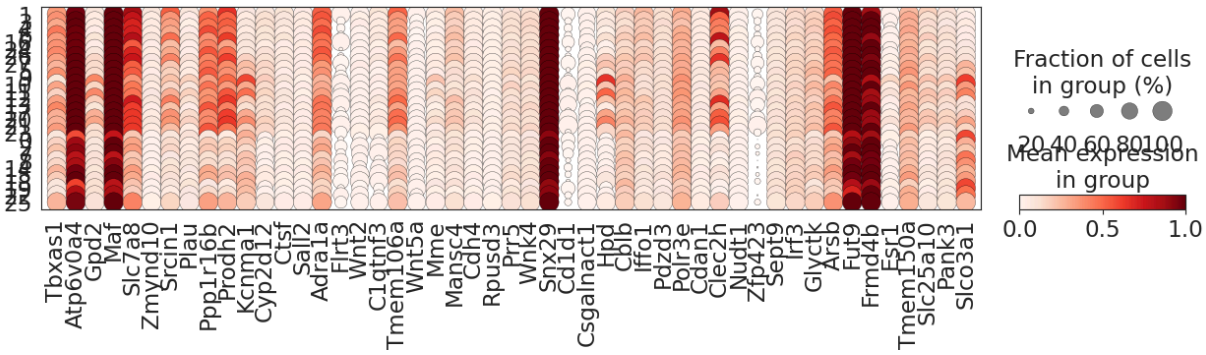
```
    # Add to the marker_genes_dict
    marker_genes_dict[group] = top_genes

# Optional: Handle any specific group logic (like 'PT-S12' in the original script)
# For example, if you want to include the top 50 genes for "PT_S1"

# Plotting
plt.rcParams.update({'font.size': 20})

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(15, 8), gridspec_kw={'wspace': 0.5, '
#cluster_order = [str(i) for i in range(28)]  # This creates a list of strings from
cluster_order = ['1', '3', '4', '6', '16', '24', '26', '27','5', '9', '10', '11', '

# Create dotplots for each group
sc.pl.dotplot(oracle.adata, marker_genes_dict['PT_S1'], 'louvain', layer="imputed_c
              dendrogram=False, ax=ax1, figsize=(10, 4), show=False, categories_ord
sc.pl.dotplot(oracle.adata, marker_genes_dict['PT_S1'], 'louvain', layer="simulated
              vmax=1, dendrogram=False, ax=ax2, figsize=(10, 4), show=False, catego

# Save the plots
plt.savefig(f"{save_folder}/{goi}_dotplot_top50genes_PT_S1.png")
plt.savefig(f"{save_folder}/{goi}_dotplot_top50genes_PT_S1_KO.png.pdf")
plt.show()

###By celltype
plt.rcParams.update({'font.size': 20})

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(15, 8), gridspec_kw={'wspace': 0.5, '
#cluster_order = [str(i) for i in range(28)]  # This creates a list of strings from

# Create dotplots for each group
sc.pl.dotplot(oracle.adata, marker_genes_dict['PT_S1'], 'celltype', layer="imputed_
              dendrogram=False, ax=ax1, figsize=(10, 4), show=False)
sc.pl.dotplot(oracle.adata, marker_genes_dict['PT_S1'], 'celltype', layer="simulate
              vmax=1, dendrogram=False, ax=ax2, figsize=(10, 4), show=False)

# Save the plots
plt.savefig(f"{save_folder}/{goi}_dotplot_top50genes_PT_S1_celltype.png")
plt.savefig(f"{save_folder}/{goi}_dotplot_top50genes_PT_S1_celltype_KO.png.pdf")
plt.show()
```

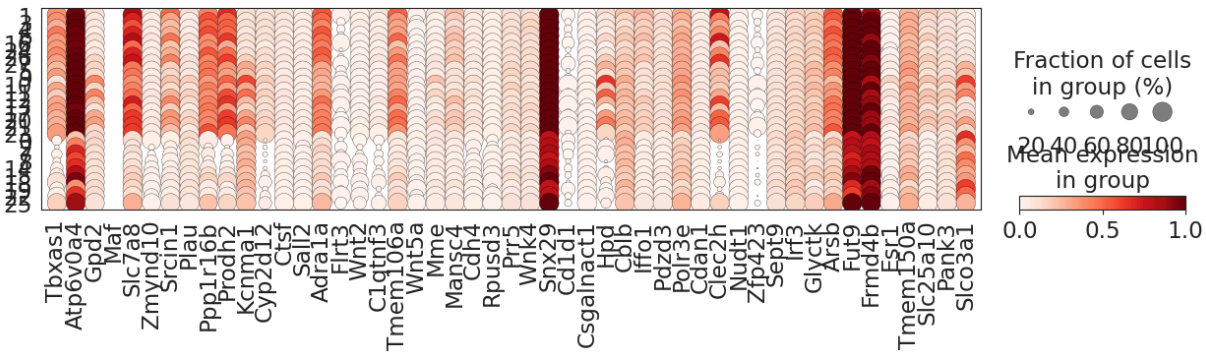


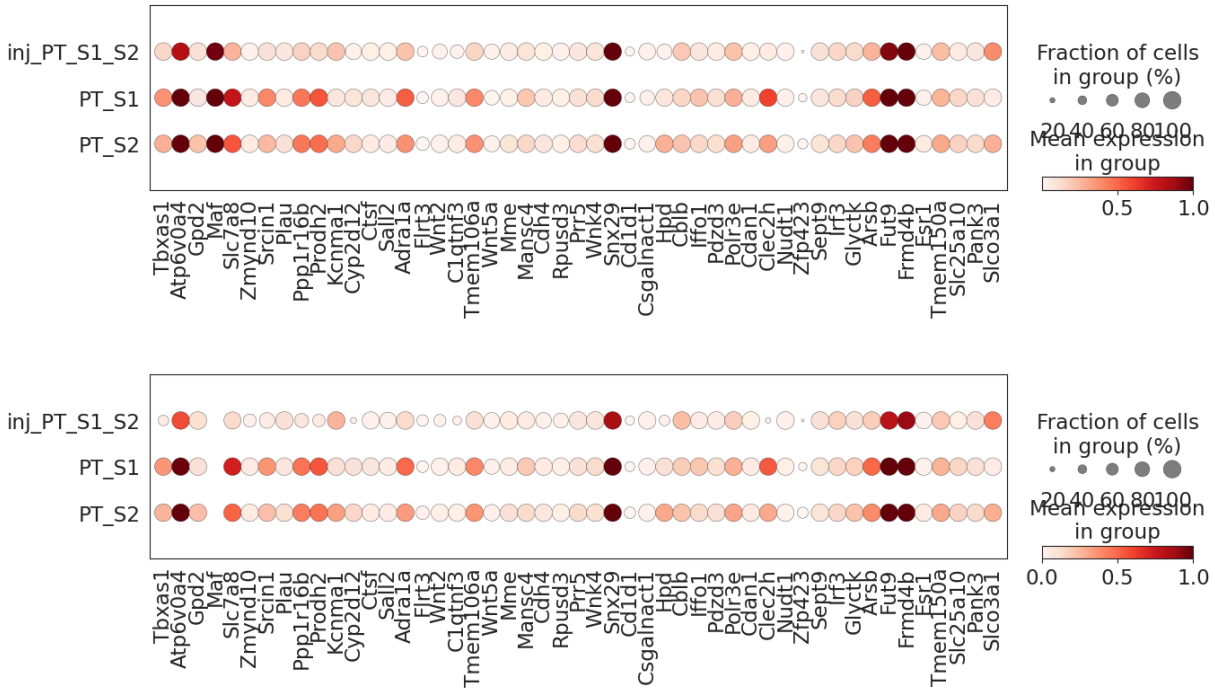
In [29]:

```
# Plotting
plt.rcParams.update({'font.size': 20})

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(15, 8), gridspec_kw={'wspace': 0.5, '
#cluster_order = [str(i) for i in range(28)]  # This creates a list of strings from
cluster_order = ['1', '3', '4', '6', '16', '24', '26', '27','5', '9', '10', '11', '

# Create dotplots for each group
sc.pl.dotplot(oracle.adata, marker_genes_dict['PT_S2'], 'louvain', layer="imputed_c
              dendrogram=False, ax=ax1, figsize=(10, 4), show=False, categories_ord
sc.pl.dotplot(oracle.adata, marker_genes_dict['PT_S2'], 'louvain', layer="simulated
              vmax=1, dendrogram=False, ax=ax2, figsize=(10, 4), show=False, catego
```

```
# Save the plots
plt.savefig(f"{save_folder}/{goi}_dotplot_top50genes_PT_S2.png")
plt.savefig(f"{save_folder}/{goi}_dotplot_top50genes_PT_S2_KO.png.pdf")
plt.show()

###By celltype
plt.rcParams.update({'font.size': 20})

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(15, 8), gridspec_kw={'wspace': 0.5, '
cluster_order = [str(i) for i in range(28)]  # This creates a list of strings from
# Create dotplots for each group
sc.pl.dotplot(oracle.adata, marker_genes_dict['PT_S2'], 'celltype', layer="imputed_
              dendrogram=False, ax=ax1, figsize=(10, 4), show=False)
sc.pl.dotplot(oracle.adata, marker_genes_dict['PT_S2'], 'celltype', layer="simulate
              vmax=1, dendrogram=False, ax=ax2, figsize=(10, 4), show=False)

# Save the plots
plt.savefig(f"{save_folder}/{goi}_dotplot_top50genes_PT_S2_celltype.png")
plt.savefig(f"{save_folder}/{goi}_dotplot_top50genes_PT_S2_celltype_KO.png.pdf")
plt.show()
```

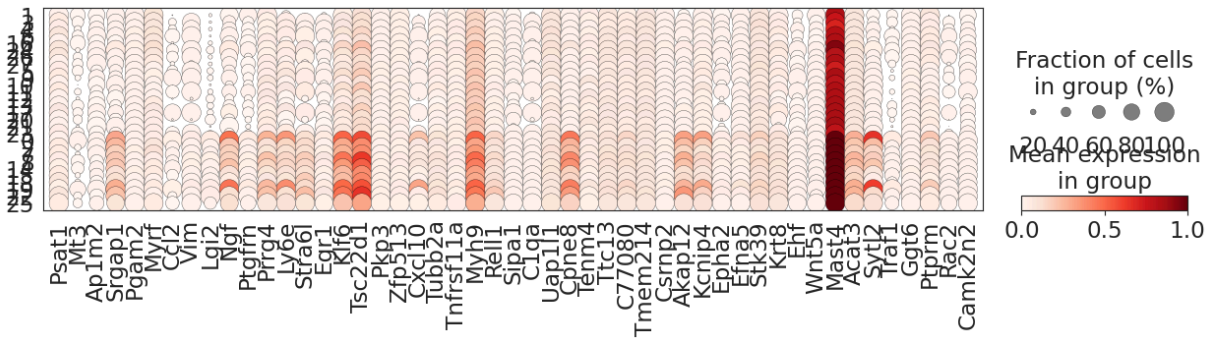


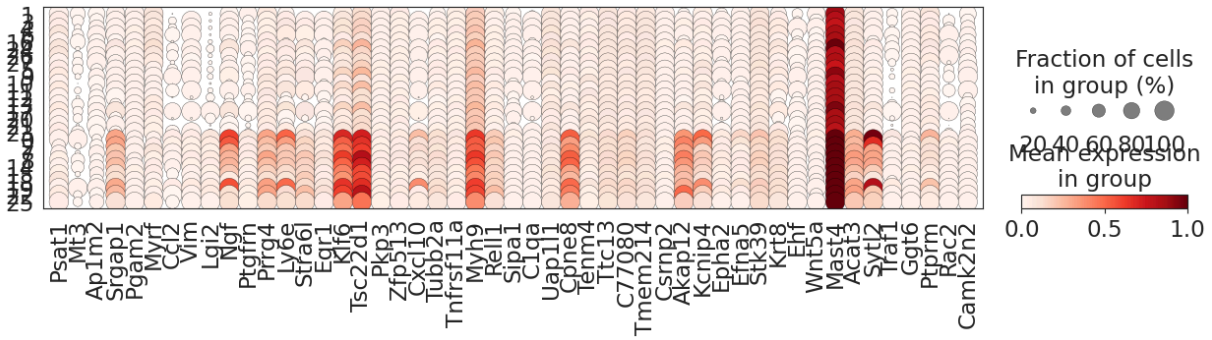


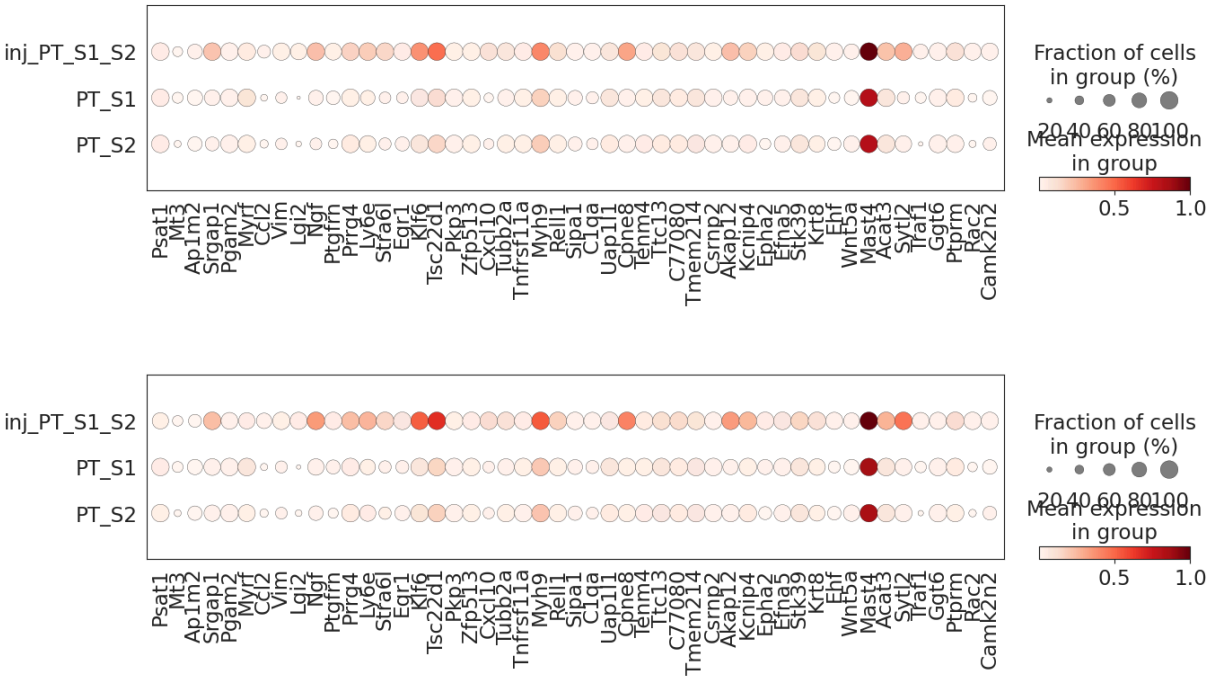


In [30]:

```
# Plotting
plt.rcParams.update({'font.size': 20})

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(15, 8), gridspec_kw={'wspace': 0.5, '
#cluster_order = [str(i) for i in range(28)]  # This creates a list of strings from
cluster_order = ['1', '3', '4', '6', '16', '24', '26', '27','5', '9', '10', '11', '

# Create dotplots for each group
sc.pl.dotplot(oracle.adata, marker_genes_dict['inj_PT_S1_S2'], 'louvain', layer="im
              dendrogram=False, ax=ax1, figsize=(10, 4), show=False, categories_ord
sc.pl.dotplot(oracle.adata, marker_genes_dict['inj_PT_S1_S2'], 'louvain', layer="si
              vmax=1, dendrogram=False, ax=ax2, figsize=(10, 4), show=False, catego

# Save the plots
plt.savefig(f"{save_folder}/{goi}_dotplot_top50genes_inj_PT_S1_S2.png")
plt.savefig(f"{save_folder}/{goi}_dotplot_top50genes_inj_PT_S1_S2_KO.png.pdf")
plt.show()

###By celltype
plt.rcParams.update({'font.size': 20})

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(15, 8), gridspec_kw={'wspace': 0.5, '
cluster_order = [str(i) for i in range(28)]  # This creates a list of strings from
# Create dotplots for each group
sc.pl.dotplot(oracle.adata, marker_genes_dict['inj_PT_S1_S2'], 'celltype', layer="i
              dendrogram=False, ax=ax1, figsize=(10, 4), show=False)
sc.pl.dotplot(oracle.adata, marker_genes_dict['inj_PT_S1_S2'], 'celltype', layer="s
              vmax=1, dendrogram=False, ax=ax2, figsize=(10, 4), show=False)

# Save the plots
plt.savefig(f"{save_folder}/{goi}_dotplot_top50genes_inj_PT_S1_S2_celltype.png")
plt.savefig(f"{save_folder}/{goi}_dotplot_top50genes_inj_PT_S1_S2_celltype_KO.png.p
plt.show()
```


Psat1 Mt3 Ap1m2 Srgap1 Pgam2 Myrf Ccl2 Vim Lgi2 Ngf Ptgfrn Prrg4 Ly6e Stra6l Egr1 Klf6 Tsc22d1 Pkp3 Zfp513 Cxcl10 Tubb2a Tnfrsf11a Myh9 Rell1 Sipa1 C1qa Uap1l1 Cpne8 Tenm4 Ttc13 C77080 Tmem214 Csmp2 Akap12 Kcnip4 Epha2 Efna5 Stk39 Krt8 Ehf Wnt5a Mast4 Acat3 Sytl2 Traf1 Ggt6 Ptprm Rac2 Camk2n2
Fraction of cells in group (%)
20 40 60 80 100
Mean expression in group
0.0 0.5 1.0


Psat1 Mt3 Ap1m2 Srgap1 Pgam2 Myrf Ccl2 Vim Lgi2 Ngf Ptgfrn Prrg4 Ly6e Stra6l Egr1 Klf6 Tsc22d1 Pkp3 Zfp513 Cxcl10 Tubb2a Tnfrsf11a Myh9 Rell1 Sipa1 C1qa Uap1l1 Cpne8 Tenm4 Ttc13 C77080 Tmem214 Csmp2 Akap12 Kcnip4 Epha2 Efna5 Stk39 Krt8 Ehf Wnt5a Mast4 Acat3 Sytl2 Traf1 Ggt6 Ptprm Rac2 Camk2n2
Fraction of cells in group (%)
20 40 60 80 100
Mean expression in group
0.0 0.5 1.0


inj_PT_S1_S2
PT_S1
PT_S2
Psat1 Mt3 Ap1m2 Srgap1 Pgam2 Myrf Ccl2 Vim Lgi2 Ngf Ptgfrn Prrg4 Ly6e Stra6l Egr1 Klf6 Tsc22d1 Pkp3 Zfp513 Cxcl10 Tubb2a Tnfrsf11a Myh9 Rell1 Sipa1 C1qa Uap1l1 Cpne8 Tenm4 Ttc13 C77080 Tmem214 Csmp2 Akap12 Kcnip4 Epha2 Efna5 Stk39 Krt8 Ehf Wnt5a Mast4 Acat3 Sytl2 Traf1 Ggt6 Ptprm Rac2 Camk2n2
Fraction of cells in group (%)
20 40 60 80 100
Mean expression in group
0.5 1.0

inj_PT_S1_S2
PT_S1
PT_S2
Psat1 Mt3 Ap1m2 Srgap1 Pgam2 Myrf Ccl2 Vim Lgi2 Ngf Ptgfrn Prrg4 Ly6e Stra6l Egr1 Klf6 Tsc22d1 Pkp3 Zfp513 Cxcl10 Tubb2a Tnfrsf11a Myh9 Rell1 Sipa1 C1qa Uap1l1 Cpne8 Tenm4 Ttc13 C77080 Tmem214 Csmp2 Akap12 Kcnip4 Epha2 Efna5 Stk39 Krt8 Ehf Wnt5a Mast4 Acat3 Sytl2 Traf1 Ggt6 Ptprm Rac2 Camk2n2
Fraction of cells in group (%)
20 40 60 80 100
Mean expression in group
0.5 1.0

In [ ]: